Database Development and Design (SOE201)

Lab 1: Tutorial on Database Storage

Dr. Wei Wang

AIAT

Q1

- Given a disk with the following characteristics:
  - There are 2^{14}=16,384 tracks per surfaces
  - There are 2^{7}=128 sectors per track
  - There are 2^{12}=4,096 byte per sector
  - The disk rotates at 7,200rpm (rounds per minute); i.e. it makes one rotation in 8.33 milliseconds
  - To move the head arm between cylinders (tracks) take one milliseconds to start and stop, plus one additional millisecond for every 1,000 cylinders travelled.
- Questions:
  - 1. what is the time to take one track movement?
  - 2. what is the time to move the head from innermost track to outmost track?

Q1 Answer

- *The disk rotates at 7,200rpm; equivalent to 120 rounds per second; then one round would take 1/120=0.00833 second; i.e. it makes one rotation in 8.33 milliseconds.*

- (1) 1.001ms: move head arm between tracks needs 1 ms plus 0.001ms for cylinder travelling
- (2) 17.384 millisecond: 16384 tracks on a surface, so total cylinder travelling is 16.384ms, plus 1 ms, which is 17.384ms

Q2

- Given a disk with the following characteristics:
  - There are 2^{14}=16384 tracks per surfaces
  - There are 2^{7}=128 sectors per track
  - There are 2^{12}=4096 byte per sector
  - The disk rotates at 7200rpm; i.e., it makes one rotation in 8.33 milliseconds
  - To move the head arm between cylinders (tracks) takes one milliseconds to start and stop, plus one additional millisecond for every 1000 cylinders travelled.
- Questions:
  - 1. Assume that there is no gap between sectors and each block occupies 4 sectors. what is the minimum time to read a block ?
  - 2. What is the maximum time to read a block?
  - 3. What is the average time?

Q2 Answer

- (1) Best case: seek time = 0; rotational latency = 0
  - Time taken for one complete rotation: 1/(7200/60) = 0.00833s = 8.33ms
  - Reading 4 sectors only needs to rotate 4/128
  - *So the time is 8.33*4/128 = 0.2603 ms*

Q2 Answer cont'd

- (2) Worst case:
  - Worst time: travel from innermost to outermost track; then rotate the whole track (*assume the read/write head passes the first sector a little*); then plus the read time.
  - 1 + 16.38 + 8.33 + 0.2603 (rotation latency: 8.33; transfer time: 0.2603)
- (3) Average case:
  - Average time is "1/2" of the worst time except for the actual read time
  - (17.38+8.33)/2 + 0.2603

Q3

- Suppose that a relation called student holds 25,000 tuples, which are stored as fixed length and fixed format records. The length of each tuple is 350 bytes. The key attribute, student_ID, occupies 10 bytes and another attribute address occupies 50 bytes. The records are sequentially ordered by student_ID and stored in a number of blocks. Each block has the size of 4,096 bytes (i.e. 4 Kilobytes). Assume that a complete record or an index entry must be stored in one block.
  - (1) How many blocks are needed to store the relation student?
  - (2) Consider creating a primary index on the *student_ID* attribute. Each index entry contains a search key and a 10-byte long pointer to the records. Suppose the primary index is sparse (i.e. one index entry for one block), compute the number of blocks needed to store the index.

Q3 Answers

- (1)
  - Each tuple of student is 350 bytes. Each block at most holds $\lfloor 4{,}096 \text{ bytes} / 350 \text{ bytes} \rfloor = 11$ tuples (where $\lfloor\ \rfloor$ indicates round down).
  - There are 25,000 tuples, so $\lceil 25000 \text{ tuples} / 11 \text{ tuples per block} \rceil = 2{,}273$ blocks required (where $\lceil\ \rceil$ indicates round up).
- (2)
  - Each index entry is 10 bytes for the key plus 10 bytes for the pointer (20 bytes in total).
  - Each block at most can store $\lfloor 4096 \text{ bytes} / 20 \text{ bytes} \rfloor = 204$ index entries.
  - There are 2,273 blocks in the *student* relation (answer from question 1), so $\lceil 2{,}273 / 204 \rceil = 12$ blocks are needed.

NOTE: answers that do not use round up or round down functions will be considered as wrong.

Q4

- A relational database contains the following two relations:
  - Relation *student*:
    - Tuples are stored as fixed-format, fixed-length records, each of 300 bytes.
    - Each tuple contains a key attribute *sID* of length 20 bytes; other fields and the record header make up the rest.
    - There are 1,000 tuples.
  - Relation *module*:
    - Tuples are stored as fixed-format, fixed-length records, each of 200 bytes.
    - Tuples contain attribute *module.sID* (ID of a student who takes a module), referencing the *student.sID*.
    - There are 5,000 tuples.
- Assume that each student must take exactly 5 modules. The student records are sequentially ordered by *sID* and the "clustered disk organisation" strategy is used. This means that, for each student record, the 5 module records also reside on the same block. Also, assume that a record does not span over more than one block, and the size of each block is 4,096 bytes.

Q4 – cont'd

- (1) Calculate the number of disk blocks needed to store the two relations.
- (2) Suppose that a dense primary index is to be created on *sID* for the *student* relation, i.e. one index entry for each tuple. Each index entry includes a key and a 10-byte pointer to data (an 8 byte block ID and a 2 byte offset). How many disk blocks are needed to store the index?
- (3) Suppose that a sparse primary index is to be created on *sID* for the *student* relation, i.e. one index entry for each disk block. Each index entry includes a key and a 10-byte pointer to data (an 8 byte block ID and a 2 byte offset). How many disk blocks are needed to store the index?

Q4 Answers

- (1) A student record, plus the 5 module records, has the size 300 + 5*200 = 1,300 bytes. Each block can hold $\lfloor 4{,}096/1{,}300 \rfloor$ = 3 of them. There are 1,000 student tuples, so $\lceil 1{,}000 / 3 \rceil$ = 334 blocks are needed to store both relations.
- (2) Clustered organisation, dense primary index: each block can hold up to $\lfloor 4{,}096/(20+10) \rfloor$ = 136 index entries. There are 1,000 tuples in the student relation, so $\lceil 1{,}000/136 \rceil$ = 8 blocks are needed.
- (3) Clustered organisation, sparse primary index: This index requires one index entry for every block. Each block can still hold 136 index entries, but now there are 334 entries needed (answer from question 1), so $\lceil 334/136 \rceil$ = 3 blocks are needed.

NOTE: answers that do not use round up or round down functions will be considered as wrong.

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Q5

- A relation named product has 10,000 tuples, which are stored as fixed length and fixed format records; each has the length of 50 bytes. Tuples contain the key attribute productID with length of 2 bytes. The tuples are stored sequentially in a number of blocks, ordered by productID. Each block has the size of 4,096 bytes and each tuple is fully contained in one block.
  - (1) What would be the number of disk blocks needed to store the relation product?
  - (2) Suppose that a sparse primary index (i.e. one index entry for one block) on productID is to be created. A 10-byte pointer to the actual tuples is needed for each index entry. Each index entry is also fully contained in one block. What would be the number of disk blocks needed to store the entire index?

Q5 Answers

- (1) Each tuple is 50 bytes. Each block holds $\lfloor 4096 \text{ bytes} / 50 \text{ bytes} \rfloor = 81$ tuples. There are 10,000 tuples, so $\lceil 10{,}000 \text{ tuples} / 81 \text{ tuples per block} \rceil = 124$ blocks required. (where $\lceil\ \rceil$ indicates round up, and $\lfloor\ \rfloor$ for round down).
- (2) Each index entry is 2+10=12 bytes. So each block can hold $\lfloor 4096 \text{ bytes} / 12 \text{ bytes} \rfloor = 341$ index entries. There are 124 blocks in the relation, so $\lceil 124/341 \rceil = 1$ block is required.

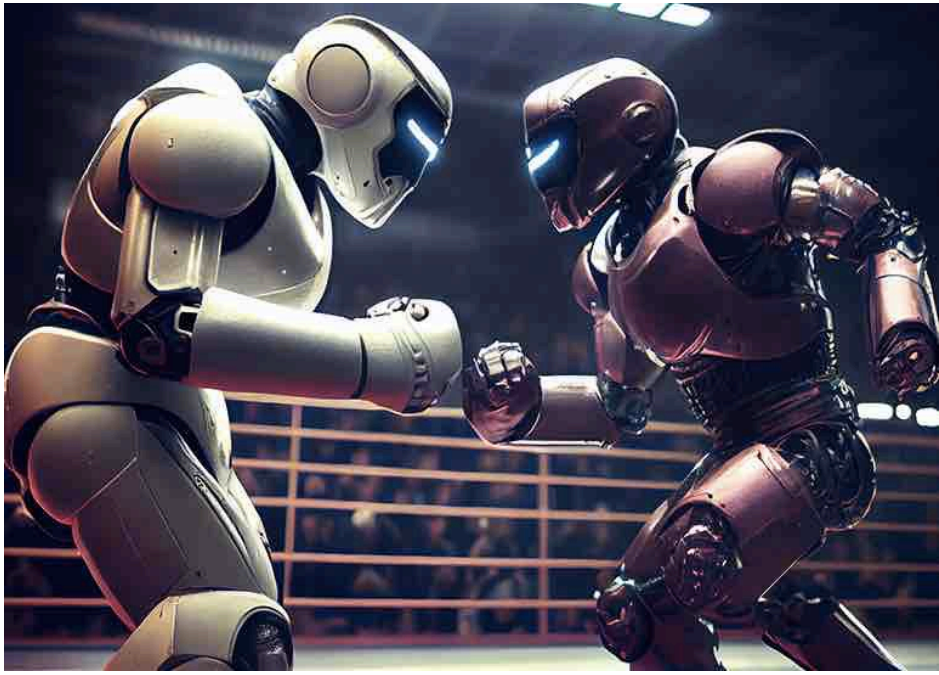
NOTE: answers that do not use round up or round down functions will be considered as wrong.

Q6

- 1) Relation students has 30,000 tuples, which are stored as fixed length and fixed format records. Each tuple has a length of 30 bytes. Tuples contain the key attribute studentID with a length of 2 bytes; a non-key attribute name with a length of 4 bytes. The tuples are stored sequentially in a number of blocks, ordered by name. Assume that each block has a size of 4,096 bytes and each tuple is fully contained in one block. What would be the number of disk blocks needed to store the relation students?
- 2) Based on the same information as in Question 1), suppose that a dense primary index is to be created. A 6-byte pointer to the actual tuple is needed for each index entry. Each index entry is also fully contained in one block. What would be the maximum number of blocks needed to store this index?
- 3) Based on the same information as in Question 1) and Question 2), suppose that a sparse secondary index on the studentID attribute is to be created. Which of the two indices, the dense primary index and sparse secondary index, is more efficient in looking-up tuples with given search keys? Justify your answer.

AI Battle for Q6

- Following AI models answered Q6
  - DeepSeek V4.1 Flash
  - GPT-5.6 Luna
  - Claude Sonnet 5
  - Gemini 3.6 Flash



All models tested on *12 September 2026*

Image source: https://www.flyingsolo.com.au/business-technology/technology/googles-bard-vs-chatgpt-the-ai-battle-of-the-year/

Xi'an Jiaotong-Liverpool University 西交利物浦大学

DeepSeek V4.1 Flash

1) Number of disk blocks needed to store the relation `students`

Given:

- Number of tuples = 30,000
- Tuple size = 30 bytes
- Block size = 4,096 bytes
- Tuples are stored fully within one block (no spanning)



Calculation:

- Tuples per block = $\lfloor 4096 / 30 \rfloor = \lfloor 136.53 \rfloor =$ **136 tuples per block**
- Number of blocks = $\lceil 30{,}000 / 136 \rceil = \lceil 220.59 \rceil =$ **221 blocks**

Answer:

$$\boxed{221}$$

DeepSeek V4.1 Flash – cont'd

2) Maximum number of blocks needed for a dense primary index

Given:

- Dense primary index → one index entry per tuple in the data file
- Number of tuples = 30,000
- Each index entry contains:
  - Key: `studentID` = 2 bytes
  - Pointer: 6 bytes
  - Total entry size = 2 + 6 = **8 bytes**
- Each index entry fully contained in one block
- Block size = 4,096 bytes



Calculation:

- Index entries per block = ⌊4096 / 8⌋ = 512 entries per block
- Number of index blocks = ⌈30,000 / 512⌉ = ⌈58.59⌉ = **59 blocks**

Answer:

$$\boxed{59}$$

DeepSeek V4.1 Flash – cont'd

3) Efficiency: dense primary index vs sparse secondary index on `studentID`

We need to consider:

- **Dense primary index**:
  - One entry per tuple
  - The data file is ordered by `name`, not `studentID` in Question 1 (it says ordered by `name`).
  - Wait — but a *primary index* usually means the data file is ordered by the index key. However, here the file is ordered by `name`, and `studentID` is the key attribute — but it's not the ordering field. So a **dense primary index** doesn't match the file ordering — this is unusual. Typically, a primary index is on the ordering key. So maybe they mean "dense index on studentID" but it's called primary because studentID is the primary key. In that case, it's a dense index but not a clustering index (data not sorted by studentID).
- **Sparse secondary index on `studentID`**:
  - Secondary index means data file is not ordered by `studentID`.
  - Sparse means one index entry per block of data file (not per tuple), pointing to the first tuple in each block with that key range.
  - But for a secondary index, a sparse index only makes sense if the data is ordered by the indexing field — otherwise you can't have "one per block" reliably unless it's a clustering secondary index.
  - If the file is not ordered by `studentID`, a sparse index on `studentID` is problematic because one entry per block doesn't guarantee all tuples with a given `studentID` are in one block (since duplicates possible? studentID is key so unique, but still scattered). For unique keys in unsorted file, sparse secondary index is not feasible — you'd need dense.

Given the file is ordered by `name`, and `studentID` is unique but not sorted, the only way to index `studentID` is a **dense index** (either primary or secondary). A sparse index on `studentID` would not work unless the file were sorted by `studentID`.

But they ask: which is more efficient in looking up tuples with given search keys?

- **Dense primary index on studentID**: direct access to the tuple via pointer → 1 index lookup + 1 data block access.
- **Sparse secondary index on studentID**: if it existed, it would point to a block, then you'd search within the block. But since studentID is unique, a sparse index would point to the block containing that studentID's tuple only if you had one entry per distinct studentID — that's dense. Sparse would mean fewer entries, but then you can't directly find unique studentID without scanning.

So the **dense primary index is more efficient** for equality search on studentID because it gives direct tuple address, whereas a sparse secondary index (if possible) would require scanning within a block and possibly multiple blocks if the file isn't sorted by studentID.

Answer:

$$\boxed{\text{Dense primary index is more efficient}}$$

Reason: It provides a direct pointer to the exact tuple for a given `studentID`, requiring one index access and one data block access. A sparse secondary index on a non-ordering key would not guarantee direct access and would require additional searching within blocks, making it less efficient for point queries.

GPT-5.6 Luna

ChatGPT

1) Number of disk blocks for the `students` relation

Given:

- Number of tuples = **30,000**
- Tuple size = **30 bytes**
- Block size = **4,096 bytes**
- Each tuple must be fully contained within one block.

The number of tuples that can fit in one block is:

$$\left\lfloor \frac{4096}{30} \right\rfloor = \lfloor 136.53 \rfloor = 136$$

So, each block can store **136 tuples**.

Therefore, the number of blocks required is:

$$\left\lceil \frac{30,000}{136} \right\rceil = \lceil 220.588 \rceil = \boxed{221}$$

Answer: 221 disk blocks.

GPT-5.6 Luna – cont'd

ChatGPT

2) Maximum number of blocks for the dense primary index

A **dense index** has one index entry for every tuple.

Thus:

$$30,000 \text{ index entries}$$

Each index entry consists of:

- `studentID` = 2 bytes
- Pointer to actual tuple = 6 bytes

Therefore, index-entry size is:

$$2 + 6 = 8 \text{ bytes}$$

The number of index entries that fit into one block is:

$$\left\lfloor \frac{4096}{8} \right\rfloor = 512$$

Hence the number of index blocks required is:

$$\left\lceil \frac{30,000}{512} \right\rceil = \lceil 58.594 \rceil = \boxed{59}$$

Answer: 59 disk blocks.

> Note: Strictly speaking, because the relation is ordered by `name`, the **primary index would normally be on the ordering attribute (`name`)**, not `studentID`. However, the calculation above follows the question's stated index-entry information.

GPT-5.6 Luna – cont'd

ChatGPT

3) Dense primary index vs. sparse secondary index

There is an important issue here.

The relation is physically ordered by `name`, whereas the proposed secondary index is on `studentID`.

A **sparse index** generally requires the data file to be ordered according to the index search key. Since the records are ordered by `name` rather than `studentID`, a genuinely sparse index on `studentID` **cannot normally be used**.

Therefore, under the standard database-index definition:

$$\boxed{\text{A sparse secondary index on studentID is not valid for this file organization.}}$$

A secondary index on `studentID` would normally need to be **dense**, because the tuples are not ordered by `studentID`.

If the question intends you to **hypothetically compare** a sparse index with the dense index, then the sparse index would be much smaller. There is approximately one index entry per data block:

$$221 \text{ entries}$$

At 8 bytes per entry:

$$\left\lceil \frac{221}{512} \right\rceil = 1$$

so it would require only **1 block** at the leaf/index level.

However, **size is not the same as lookup efficiency**. Because the data is not ordered by `studentID`, that sparse index cannot reliably identify the location of an arbitrary `studentID`.

Claude Sonnet 5

Claude

1) Number of blocks for the relation

Tuples per block = ⌊4096 / 30⌋ = ⌊136.53⌋ = **136 tuples/block**

Number of blocks = ⌈30,000 / 136⌉ = ⌈220.59⌉ = **221 blocks**

Claude Sonnet 5 – cont'd

Claude

2) Number of blocks for the dense primary index

Each index entry = search-key length + pointer length = 2 bytes (studentID) + 6 bytes (pointer) = **8 bytes/entry**

Entries per block = ⌊4096 / 8⌋ = **512 entries/block**

Since the index is *dense*, there is one index entry for every tuple in the relation (30,000 entries):

Number of index blocks = ⌈30,000 / 512⌉ = ⌈58.59⌉ = **59 blocks**

Claude Sonnet 5 – cont'd

Claude

3) Dense primary index vs. sparse secondary index — which is more efficient?

The dense primary index is more efficient (and, in fact, the only one that works correctly here).

Justification:

- A **sparse index** only works correctly when the data file is physically **sorted on the index's search key**, because a sparse index stores only one entry per block (e.g., the first key value in that block) and relies on the fact that all other keys in that block fall in a known range relative to neighboring blocks. The lookup algorithm then scans sequentially within the target block to find the exact record.
- In this scenario, the relation `students` is stored **sorted by** `name`, not by `studentID`. So an index built on `studentID` is a **secondary index** — the key order of the index does *not* match the physical order of the file.
- If such a secondary index is made **sparse**, it cannot reliably locate a tuple: knowing "block X starts with some studentID value" tells you nothing about where a *particular* studentID is, because studentID values are scattered randomly (in name order) across blocks rather than clustered in ranges. You would effectively have to scan most or all of the relation to be sure you've found (or not found) a given studentID — defeating the purpose of an index.
- A **dense index**, by contrast, has **one entry for every tuple**, each with a direct pointer to that tuple's exact block/location. Regardless of how the data file itself is ordered, a dense index lookup is a simple search (e.g., binary search) on the index file followed by **one direct disk access** to fetch the tuple — cost ≈ $O(\log_2(\text{number of index blocks})) + 1$ I/O.

Conclusion: Because the file is ordered by `name` and not by `studentID`, only a **dense index on studentID** can correctly and efficiently retrieve a tuple by studentID in roughly log(index size) + 1 disk I/Os. The sparse secondary index is unreliable/inefficient here since it would require additional (potentially exhaustive) searching through unordered data to locate the correct tuple.

Gemini 3.6 Flash

1) Number of disk blocks needed to store the relation `students`

- Tuples per block:

$$\lfloor 4096/30 \rfloor = 136 \text{ tuples/block}$$

- Total blocks needed:

$$\lceil 30000/136 \rceil = \mathbf{221} \text{ blocks}$$

Gemini 3.6 Flash – cont'd

Gemini

2) Maximum number of blocks needed to store the dense primary index

- **Index entry size:**
  The search key (`name`) is 4 bytes + pointer (6 bytes) = **10 bytes/entry**.
- **Entries per index block:**

$$\lfloor 4096/10 \rfloor = 409 \text{ entries/block}$$

- **Total index blocks needed:**
  A dense index contains one entry for every record in the data file (30,000 entries).

$$\lceil 30000/409 \rceil = \mathbf{74} \text{ blocks}$$

Gemini 3.6 Flash – cont'd

Gemini

3) Comparison: Dense primary index vs. Sparse secondary index

First, let's correct a premise in the question:

1. **Primary Index:** A primary index is built on the ordering key of a file. Since the relation is ordered by `name`, a primary index must be built on `name`, not `studentID`.
2. **Sparse Index Condition:** A sparse index can **only** be created on the primary ordering attribute (so that every block has a defined key range). A secondary index on a non-ordering attribute (`studentID`) **must be dense**, as data blocks are not sorted by `studentID`.

Assuming the question refers to a **dense primary index on** `name` vs. a **dense secondary index on** `studentID` (or a sparse primary index on `name` vs. a dense secondary index on `studentID`):

The dense primary index is more efficient for the following reasons:

- **Contiguous Disk Locality:** A primary index points to data that is physically sorted on disk. Accessing consecutive records requires minimal sequential disk block reads.
- **Secondary Index Overhead:** A secondary index on `studentID` must be dense. Because the physical data file is ordered by `name` rather than `studentID`, consecutive index entries point to arbitrary data blocks across the disk, leading to random I/O operations for range queries or non-unique lookups.
- **Sparse Index Infeasibility:** A sparse index cannot be built on a non-ordering key like `studentID` without sorting the file by `studentID` first.

AI Battle for Q6 - Conclusions

- Gemini 3.6 Flash knocked out all others for Q6 (very easy question);
- But might NOT be case for other questions;
- Important thing is to recognise errors (perhaps hallucination isn't good term here);
- Again, always use AI with CAUTIONS!

Q6 Answers

- 1) Each tuple is 30 bytes. Each block can hold up to $\lfloor 4{,}096 \text{ bytes} / 30 \text{ bytes} \rfloor = 136$ tuples. There are 30,000 tuples, so $\lceil 30{,}000 \text{ tuples} / 136 \text{ tuples per block} \rceil = 221$ blocks required.
- It is a dense primary index, so the search key is the name attribute. Each index entry is 10 bytes (4 bytes for the name attribute plus 6 bytes for the pointer), so each block can hold up to $\lfloor 4{,}096 \text{ bytes} / 10 \text{ bytes} \rfloor = 409$ index entries. There are 30,000 tuples in the relation, so $\lceil 30{,}000 / 409 \rceil = 74$ blocks are required maximum.
- The dense primary index is more efficient or useful. This is because secondary index cannot be sparse, it is not useful in looking-up tuples.

Xi'an Jiaotong-Liverpool University 西交利物浦大学